SIMATS ENGINEERING

CSA4712 – DEEP LEARNING

CO1 – ASSESSMENT 1

MATHEMATICAL FOUNDATIONS OF HEALTHCARE RISK PREDICTION

Student Name	B Akshaya
Registration Number	192424336
Submission Date	29.10.2026
Scenario	Healthcare risk prediction

Assignment overview

This assignment explains the mathematical and machine-learning choices behind a hospital disease-risk prediction system that uses patient age, blood pressure, glucose level, cholesterol and family history. Each task is presented as connected technical discussion with a worked calculation or data table, a Python code excerpt and a notebook-style output panel. The code excerpts can be executed in Google Colab. Panels for Tasks 1 to 6 and 9 are computed directly from the numbers given in the questions; panels for Tasks 7, 8 and 10 to 12 come from clearly labelled synthetic simulations and are not hospital data. All predictions are decision support for clinicians and never a diagnosis.

TASK 1 — Patient data representation and normalization

Three patients are stored as rows and four features (age, blood pressure, glucose, cholesterol) as columns, so X is a 3 × 4 matrix, X = [[40,120,100,180], [55,150,160,240], [70,135,220,210]]. **(a)** Without centering or normalization every feature keeps its own unit and range, so the feature with the largest numerical differences dominates Euclidean distance and gradient size. For P1 and P2 the raw differences are (15, 30, 60, 60); the squared distance is 225 + 900 + 3600 + 3600 = 8325 and d = 91.24, of which glucose and cholesterol supply 86.5% and age only 2.7%, although age is not clinically 32 times less informative. **(b)** Min–max scaling uses $x' = (x - \min)/(\max - \min)$ column by column, with ranges 30 (age), 30 (BP), 120 (glucose) and 60 (cholesterol). The result is X′ = [[0, 0, 0, 0], [0.5, 1, 0.5, 1], [1, 0.5, 1, 0.5]]. Verification: every column has minimum 0 and maximum 1, and the P1–P2 distance becomes $\sqrt{(0.25 + 1 + 0.25 + 1)} = 1.581$, now shared 10%, 40%, 10%, 40% instead of being controlled by units. **(c)** K-means and k-NN compare patients through summed squared coordinate differences, so unscaled units decide similarity. Gradient descent, $\theta \leftarrow \theta - \eta \cdot$(gradient of L), meets an elongated loss surface when inputs differ in scale, forcing a small learning rate and zig-zag steps; scaled inputs give balanced gradients and faster, more stable convergence.

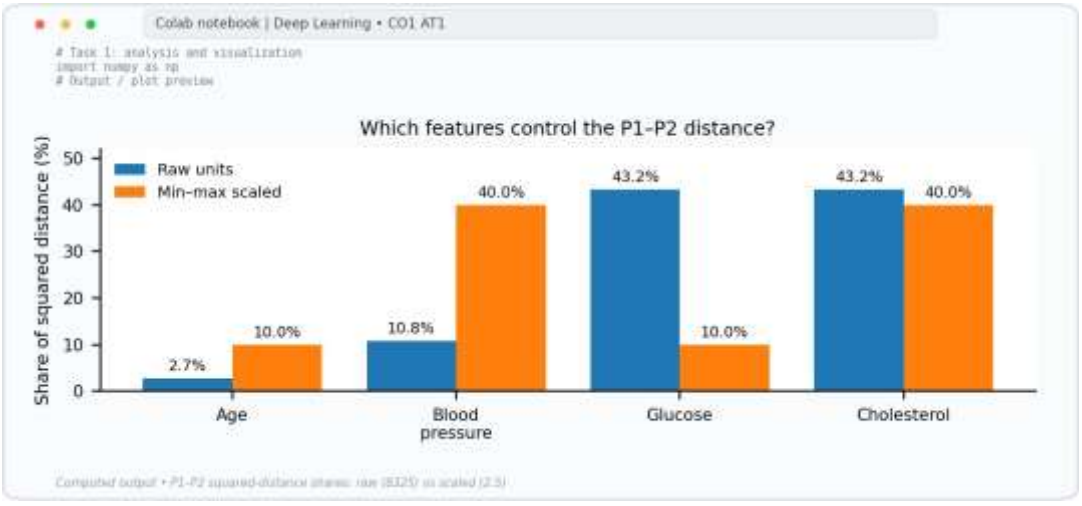
Feature matrix and normalization

Method and analysis. Compute min and max on the training split only, then apply the same constants to validation, test and future patients. Z-score standardization, $z = (x - \mu)/\sigma$, is the alternative when outliers are present.

Interpretation. Scaling removes unit effects but does not decide clinical importance; min–max values are sensitive to extreme readings, and tree models are less scale-sensitive than distance- or gradient-based models.

Patient	Raw (age, BP, glucose, cholesterol)	Normalized (age′, BP′, glucose′, cholesterol′)
P1	40, 120, 100, 180	0, 0, 0, 0
P2	55, 150, 160, 240	0.5, 1, 0.5, 1
P3	70, 135, 220, 210	1, 0.5, 1, 0.5

```
X = np.array([[40,120,100,180],[55,150,160,240],[70,135,220,210]],  dtype=float)
Xn = (X - X.min(axis=0)) / (X.max(axis=0) - X.min(axis=0))
print(Xn);  print(np.linalg.norm(X[0]-X[1]),  np.linalg.norm(Xn[0]-Xn[1]))
```



TASK 2 — Vector representation and matrix operations

Take the feature order age, glucose, blood pressure, cholesterol, with values already scaled to [0, 1]. **(a)** One patient is the vector $x = [x_{age}, x_{glu}, x_{BP}, x_{chol}]^T$ and the importance weights are $w = [w_{age}, w_{glu}, w_{BP}, w_{chol}]^T$. The preliminary risk score is the dot product $s = w^T x = \Sigma_j w_j x_j$, optionally plus an intercept b. For n patients the same rule is the matrix–vector product s = Xw, where X is n × 4, w is 4 × 1 and s is n × 1. **(b)** Let $x = [0.60, 0.90, 0.50, 0.40]^T$ and $w = [0.15, 0.40, 0.25, 0.20]^T$, so the weights sum to 1. Then s = 0.15(0.60) + 0.40(0.90) + 0.25(0.50) + 0.20(0.40) = 0.090 + 0.360 + 0.125 + 0.080 = 0.655. The score is moderately high on a 0–1 scale, and glucose supplies 0.360/0.655 = 55% of it because it has both the largest weight and a high value. Applying Xw to the three normalized patients of Task 1 (reordered as age, glucose, BP, cholesterol) gives scores 0, 0.725 and 0.775.

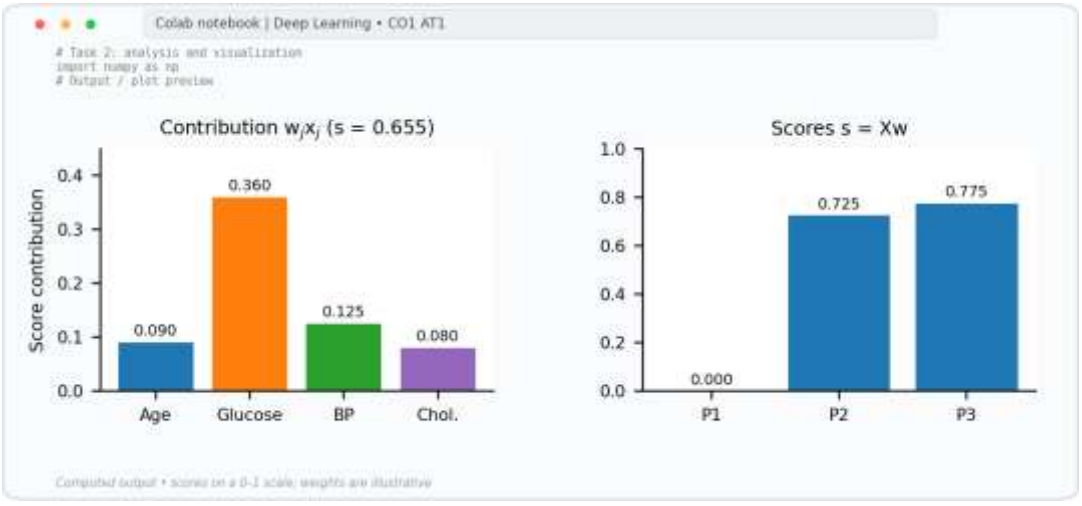
Weighted-score worked example

Method and analysis. The dot product multiplies each feature by its importance and adds the products. Because features are scaled, a weight can be read as the maximum score a feature can add; the matrix form lets a linear-algebra library score a whole ward at once.

Interpretation. The score is a ranking aid, not a probability or diagnosis. The weights here are illustrative and in practice must be agreed clinically or learned from data and validated.

Feature	Value x	Weight w	Product wx
Age	0.60	0.15	0.090
Glucose	0.90	0.40	0.360
Blood pressure	0.50	0.25	0.125
Cholesterol	0.40	0.20	0.080
Total	—	1.00	s = 0.655

```
x = np.array([0.60, 0.90, 0.50, 0.40]); w = np.array([0.15, 0.40, 0.25, 0.20])
s = w @ x                          # 0.655
Xm = np.array([[0,0,0,0],[0.5,0.5,1,1],[1,1,0.5,0.5]])
print(s, Xm @ w)                   # [0, 0.725, 0.775]
```



TASK 3 — Probability and conditional probability

Let D be “patient has the disease” and G be “patient has elevated glucose”. The counts are D∩G = 160, D∩¬G = 40, ¬D∩G = 120 and ¬D∩¬G = 680 out of 1,000 patients. **(a)** P(G) = (160 + 120)/1,000 = 0.28. **(b)** P(D | G) = P(D∩G)/P(G) = 0.16/0.28 = 160/280 = 4/7 ≈ 0.5714. **(c)** P(G | D) = P(G∩D)/P(D) = 160/200 = 0.80. Cross-check: P(D∩G) = P(G | D)P(D) = 0.80 × 0.20 = 0.16, and 0.16/0.28 = 0.5714 agrees. The two conditional probabilities differ because their denominators differ (280 patients with elevated glucose versus 200 diseased patients). **(d)** Baseline prevalence is 20%, but among patients with elevated glucose it is 57.1%, about 2.9 times higher, so elevated glucose is a sensible trigger for earlier confirmatory testing. The same table warns that 40 of 200 diseased patients (20%) have normal glucose and would be missed by this rule, and that 120 of the 280 flagged patients are disease-free.

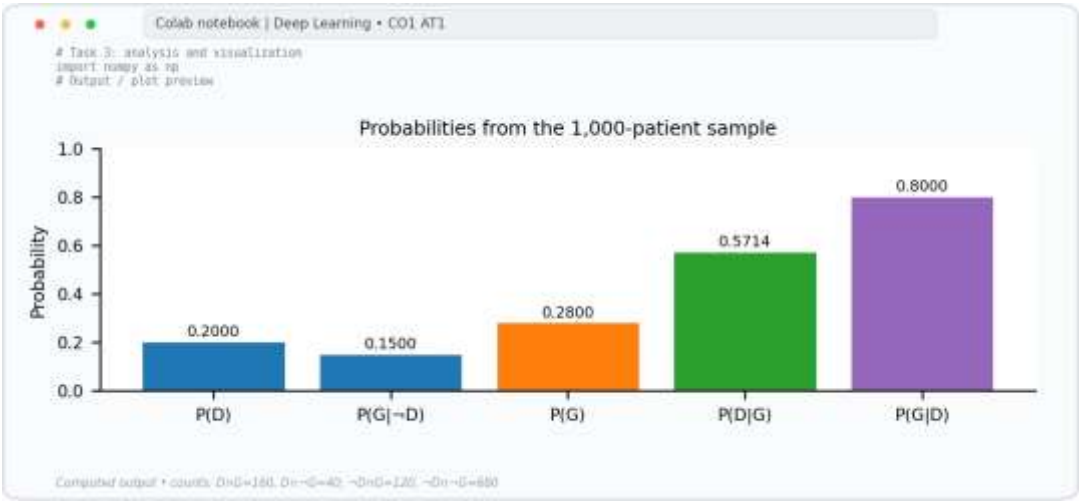
Event counts

Method and analysis. Define events, build the contingency table, then divide the relevant joint count by the count of the conditioning event. The same numbers answer all parts, which makes the arithmetic easy to verify.

Interpretation. A hospital can rank patients by conditional risk, but thresholds must weigh missed disease against false alarms, test cost and clinical guidelines; sample probabilities are estimates and not a diagnosis.

Group	Elevated glucose	Not elevated	Total
Disease (D)	160	40	200
No disease (¬D)	120	680	800
Total	280	720	1,000

```
P_G = (160 + 120) / 1000                # 0.28
P_D_given_G = 160 / (160 + 120)         # 0.5714
P_G_given_D = 160 / 200                 # 0.80
print(P_G, round(P_D_given_G, 4), P_G_given_D)
```



TASK 4 — Bayesian statistics for disease classification

Given P(D) = 0.10, P(+ | D) = 0.90 and P(+ | ¬D) = 0.05, also P(¬D) = 0.90. **(a)** By Bayes' theorem P(D | +) = P(+ | D)P(D)/P(+). The evidence is P(+) = P(+ | D)P(D) + P(+ | ¬D)P(¬D) = 0.90(0.10) + 0.05(0.90) = 0.090 + 0.045 = 0.135. Therefore P(D | +) = 0.090/0.135 = 2/3 ≈ 0.6667, i.e. 66.7%. Frequency check: in 1,000 people, 100 are diseased and 90 of them test positive; of 900 healthy people 45 test positive; 90 of the 135 positives are true, 90/135 = 66.7%. **(b)** The prior P(D) is the belief before testing; the likelihoods P(+ | D) and P(+ | ¬D) say how strongly a positive result favours each state; the evidence P(+) normalizes them; and the posterior multiplies prior by likelihood and rescales. A strong test cannot override a low prior completely: the 10% prior becomes 66.7%, not 90%. **(c)** Medical information is uncertain, tests are imperfect and prevalence differs between clinics, so Bayesian reasoning gives a principled way to combine baseline risk with each new piece of evidence and to update it sequentially.

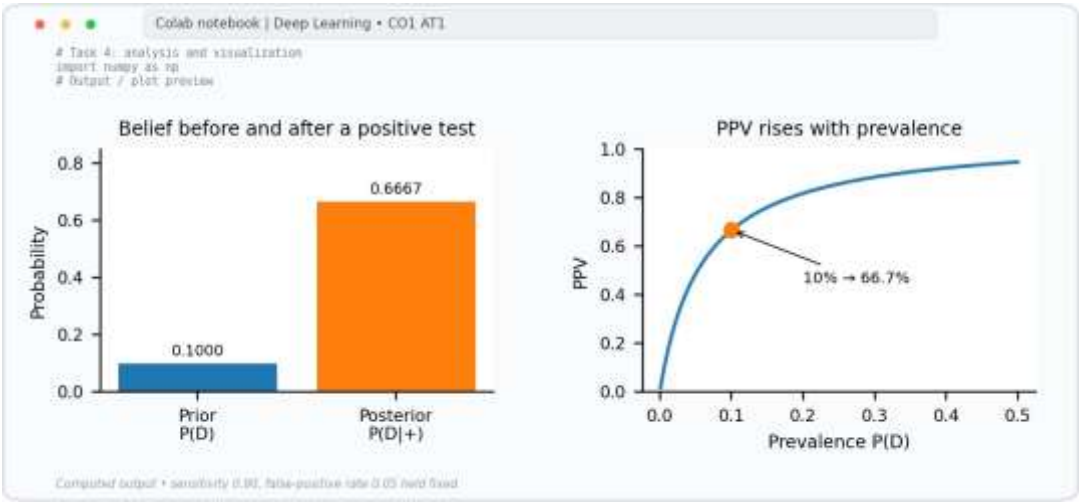
Per-1,000-patient frequency table

Method and analysis. Write the prior and conditional probabilities, compute the total probability of a positive result, then divide the true-positive share by it. Counting 1,000 imaginary patients gives the same answer without algebra.

Interpretation. Sensitivity alone does not give the chance of disease; the posterior depends strongly on prevalence, so the same test has a lower positive predictive value in a low-prevalence screening population. Correlated tests must not be multiplied as if independent.

Outcome (per 1,000 people)	Count	Comment
Disease and positive	90	0.10 × 0.90 × 1,000
Disease and negative	10	missed cases
No disease and positive	45	0.90 × 0.05 × 1,000
No disease and negative	855	correct negatives
All positive tests	135	PPV = 90/135 = 0.6667

```
pD, sens, fpr = 0.10, 0.90, 0.05
p_pos = sens*pD + fpr*(1 - pD)          # evidence = 0.135
posterior = sens*pD / p_pos
print(p_pos, round(posterior, 4))       # 0.135 0.6667
```



TASK 5 — Entropy and information gain

The 100 records contain 40 high-risk and 60 low-risk patients, so p(high) = 0.4 and p(low) = 0.6. **(a)** Entropy is $H(Y) = -\Sigma\, p_i \log_2 p_i = -[0.4 \log_2 0.4 + 0.6 \log_2 0.6] = 0.4(1.3219) + 0.6(0.7370) = 0.5288 + 0.4422 = 0.9710$ bits. **(b)** With two classes, entropy is 0 when all patients share one class and reaches the maximum of 1 bit at a 50:50 split. A value of 0.971 bits therefore means the risk label is still close to maximally uncertain, so knowing nothing else we could not predict a patient's class confidently. **(c)** Information gain is $IG(Y, X) = H(Y) - \Sigma_v P(X = v)\, H(Y \mid X = v)$, the drop in entropy after splitting on a feature. Suppose a glucose threshold splits the data into 45 high-glucose records (35 high-risk, 10 low-risk, entropy 0.764) and 55 normal-glucose records (5 high-risk, 50 low-risk, entropy 0.439). The weighted entropy is 0.45(0.764) + 0.55(0.439) = 0.586, so IG = 0.971 − 0.586 = 0.385 bits. A feature with larger gain separates the classes better and would be chosen first by a decision tree.

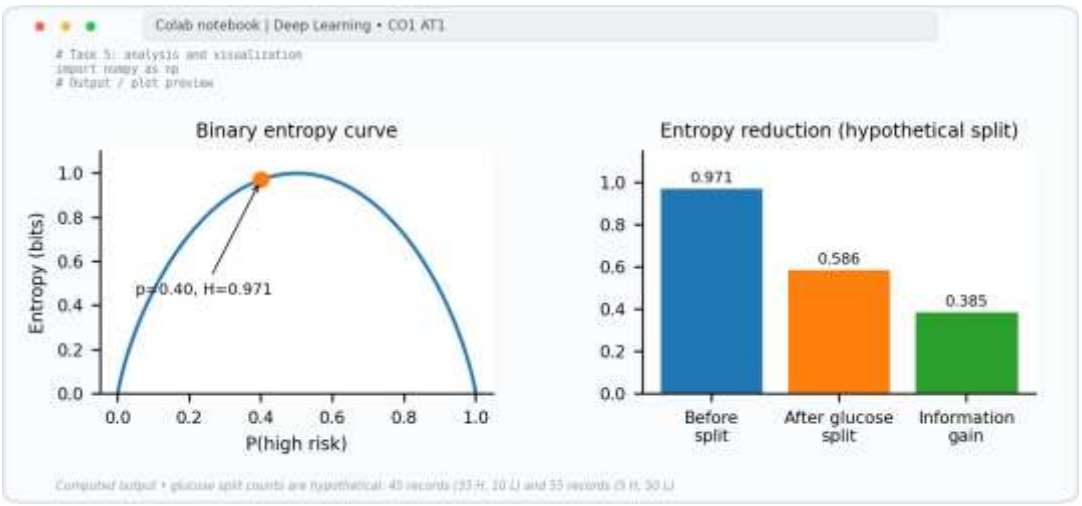
Hypothetical glucose split

Method and analysis. Compute the parent entropy, partition the records by the feature, compute each child's entropy, weight by the child's share of records and subtract. The split counts above are hypothetical and chosen only to demonstrate the method.

Interpretation. A gain of 0.385 bits means that roughly 40% of the original uncertainty is removed by glucose. Information gain measures association with the label, not causation, can favour features with many values, and must be validated on held-out data.

Node	High-risk	Low-risk	Records	Entropy (bits)
Parent	40	60	100	0.971
Glucose high	35	10	45	0.764
Glucose normal	5	50	55	0.439
Weighted after split	—	—	100	0.586
Information gain	—	—	—	0.385

```
H = lambda q: -(q*np.log2(q)).sum()
H0 = H(np.array([0.4, 0.6]))
Hs = 0.45*H(np.array([35,10])/45) + 0.55*H(np.array([5,50])/55)
print(round(H0,4), round(Hs,4), round(H0 - Hs,4))      # 0.971 0.5856 0.3853
```



TASK 6 — Numerical computation and distance measurement

Patient A = (0.70, 0.80, 0.60) and Patient B = (0.50, 0.75, 0.90) in the order glucose, blood pressure, cholesterol. **(a)** The Euclidean distance is $d(A, B) = \sqrt{\Sigma(a_j - b_j)^2} = \sqrt{[(0.70 - 0.50)^2 + (0.80 - 0.75)^2 + (0.60 - 0.90)^2]} = \sqrt{[0.0400 + 0.0025 + 0.0900]} = \sqrt{0.1325} \approx 0.3640$. **(b)** A small distance means two records have similar normalized measurements, so they would be neighbours in k-NN and would tend to fall in the same K-means cluster. In the unit cube the largest possible distance is $\sqrt{3} = 1.732$, so 0.364 is about 21% of the maximum and the two patients are fairly similar; most of the gap comes from cholesterol (0.0900 of 0.1325). Similarity in three measurements does not imply the same diagnosis, because family history, symptoms and medication are not included. **(c)** If features keep raw units, the feature with the biggest numerical range, such as glucose in mg/dL, dominates the sum of squares and the other measurements are almost ignored. Normalization puts every feature on a comparable 0–1 range so that distance reflects relative differences rather than units.

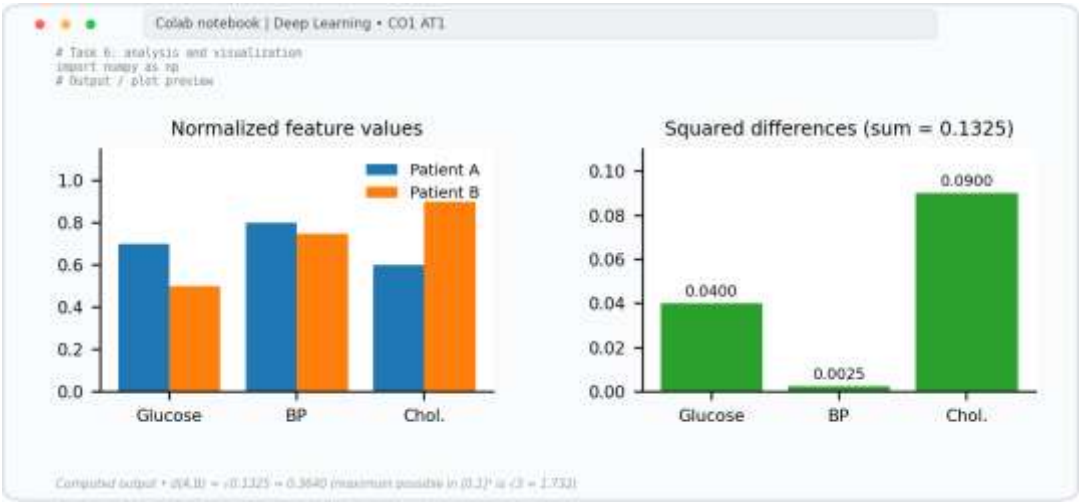
Distance contributions

Method and analysis. Subtract the vectors, square each difference, add the squares and take the square root. Reporting the squared differences shows which feature drives the distance.

Interpretation. The result is valid only in the chosen representation: after normalization, with equal feature weights and Euclidean distance. A clinically weighted distance may be preferable if some variables matter more.

Feature	Patient A	Patient B	Difference	Squared difference
Glucose	0.70	0.50	0.20	0.0400
Blood pressure	0.80	0.75	0.05	0.0025
Cholesterol	0.60	0.90	−0.30	0.0900
Total	—	—	—	0.1325 → d = 0.3640

```
A = np.array([0.70, 0.80, 0.60]); B = np.array([0.50, 0.75, 0.90])
diff = A - B
print(diff**2, diff @ diff)              # [0.04 0.0025 0.09] 0.1325
print(round(np.sqrt(diff @ diff), 4))    # 0.364
```



TASK 7 — Bias–variance trade-off

(a) Model A (high training and high validation error) has **high bias** and is underfitting; Model B (very low training error, high validation error) has **high variance** and is overfitting; Model C (low, similar errors) generalizes well. **(b)** Expected squared error decomposes into biasZ + variance + irreducible noise. A low-capacity model makes strong assumptions, so its average prediction is far from the truth (high bias) yet it changes little between training samples (low variance). As capacity grows, bias falls because the model can represent more complex relationships, but variance rises because the fitted function starts to follow the noise in the particular sample. Total error is therefore lowest at intermediate capacity. **(c)** For Model A: add informative features, interaction or nonlinear terms, use a more flexible model and reduce excessive regularization. For Model B: add regularization (L2, pruning, dropout, early stopping), reduce capacity or features, collect more representative data and tune with cross-validation. In the simulation below (noisy curve y = sin(2.2x) + 0.4x, 40 training points) degree-1, degree-9 and degree-3 polynomials play the roles of A, B and C: training/validation MSE is 0.309/0.340 for A, 0.062/0.130 for B and 0.091/0.108 for C.

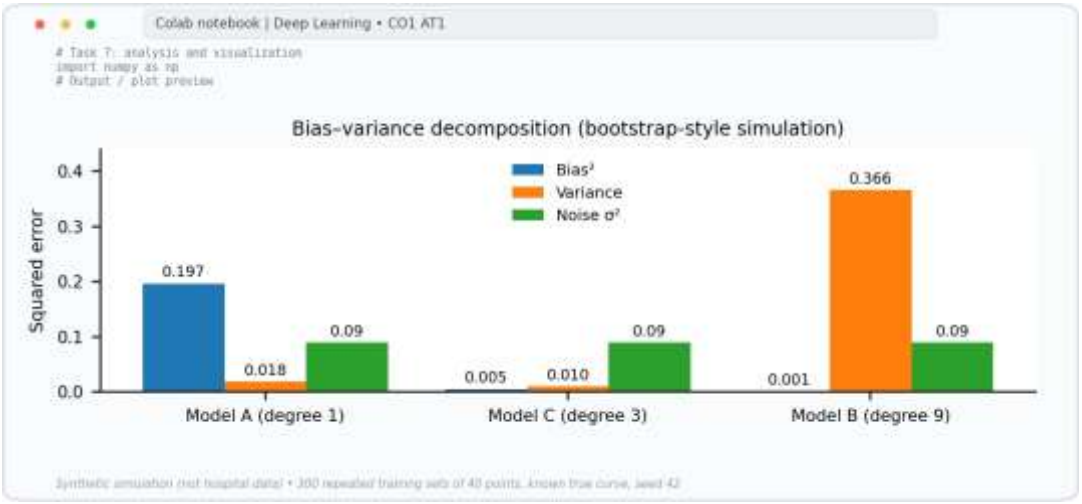
Model diagnosis

Method and analysis. A known true curve lets the decomposition be estimated: 300 independently drawn training sets are fitted at each capacity, biasZ is the squared gap between the mean prediction and the truth, and variance is the spread of predictions at fixed evaluation points. Degree 1 gives biasZ 0.197 and variance 0.018; degree 3 gives 0.005 and 0.010; degree 9 gives 0.001 and 0.366; noise variance is 0.3Z = 0.09.

Interpretation. On real patient data the true function and noise level are unknown, so the decomposition is an explanatory framework rather than an exactly observable quantity. Model C is the preferred candidate only if its validation data are independent and representative.

Model	Observed behaviour	Diagnosis	Remedy
A	High training error, high validation error	High bias, underfitting	More features or capacity, less regularization
B	Very low training error, high validation error	High variance, overfitting	Regularize, simplify, add data
C	Low and similar errors	Good generalization	Keep; validate externally

```
for deg in (1, 3, 9):
    P = [C.fit(*draw_sample(40), deg, domain=[-1.5, 1.5])(x_eval) for _ in range(300)]
    bias2 = np.mean((np.mean(P, axis=0) - f(x_eval))**2); var = np.var(P, axis=0).mean()
    print(deg, round(bias2, 3), round(var, 3))
```



TASK 8 — Overfitting, underfitting and model capacity

(a) The simple model that fails to capture disease patterns is **underfitting**; the moderately complex model that performs consistently shows **appropriate generalization**; and the highly complex model that is almost perfect on training data but poor on unseen patients is **overfitting**. **(b)** Capacity is the range of functions a model can represent. A linear model cannot express interactions such as age × glucose, whereas deeper trees, higher-degree polynomials or larger networks can represent complicated boundaries. Capacity only provides the ability to learn complex relationships; data volume, noise, regularization and validation decide whether that ability is used well. **(c)** When capacity rises from a too-small level, the model starts to capture real structure, so training and validation error both fall. Beyond the useful level the extra flexibility is spent on random fluctuations, measurement noise and site-specific artifacts: training error keeps falling, but validation error stops improving and rises. In the simulation the median validation MSE falls from 0.340 (degree 1) to 0.108 (degree 3) and then climbs to 0.130 (degree 9) while training MSE keeps decreasing to 0.062.

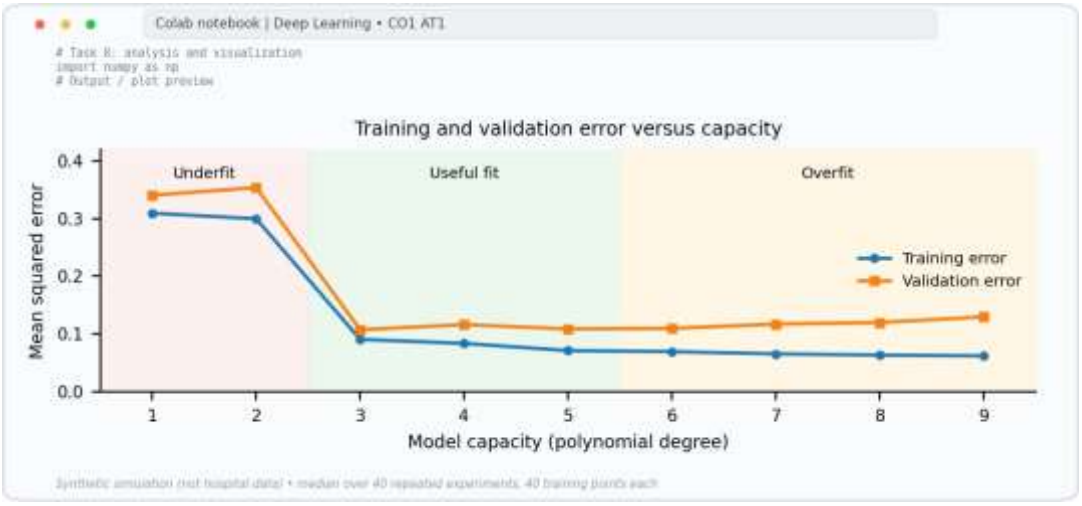
Capacity and error

Method and analysis. The model is refitted at each capacity setting (polynomial degree here; tree depth, K or regularization strength in practice), training and validation errors are stored, and both curves are plotted. The generalization gap is validation minus training error.

Interpretation. Select capacity near the minimum validation error, not the minimum training error, and reserve the test set for the final estimate. If validation error rises while training error falls, use more data, regularization, early stopping or a simpler model, and check for leakage such as the same patient appearing in both splits.

Degree	Training MSE	Validation MSE	Gap	Behaviour
1	0.309	0.340	0.031	Underfitting
3	0.091	0.108	0.017	Best generalization
5	0.071	0.109	0.038	Good fit
7	0.066	0.117	0.052	Mild overfitting
9	0.062	0.130	0.068	Overfitting

```
for deg in range(1, 10):
    m = C.fit(x_tr, y_tr, deg, domain=[-1.5, 1.5])
    print(deg, mse(y_tr, m(x_tr)), mse(y_va, m(x_va)))   # compare the two errors
```



TASK 9 — Supervised learning and model evaluation

(a) The hospital holds historical records whose outcome (diseased or disease-free) is known, and it wants to predict the outcome for new patients. Learning a mapping f: x → y from labelled input–output pairs is the definition of supervised learning; here it is binary classification, for example with logistic regression, decision trees or neural networks. **(b)** The training set is used to fit the model parameters. A validation set (or cross-validation) is used to choose hyperparameters, features and the decision threshold. The test set is kept untouched until the end to estimate performance on unseen patients; it must not leak into training, and records of one patient should stay in one split. **(c)** Accuracy = correct predictions/total predictions = 92/100 = 0.92 = 92%, so the error rate is 8%. Interpretation: the model classifies 92 of every 100 test patients correctly on this sample, but accuracy ignores which errors occur. If only 5% of patients have the disease, a useless model that always answers "disease-free" scores 95% while detecting nobody, so sensitivity, specificity, precision, F1, the confusion matrix and calibration must be reported with accuracy.

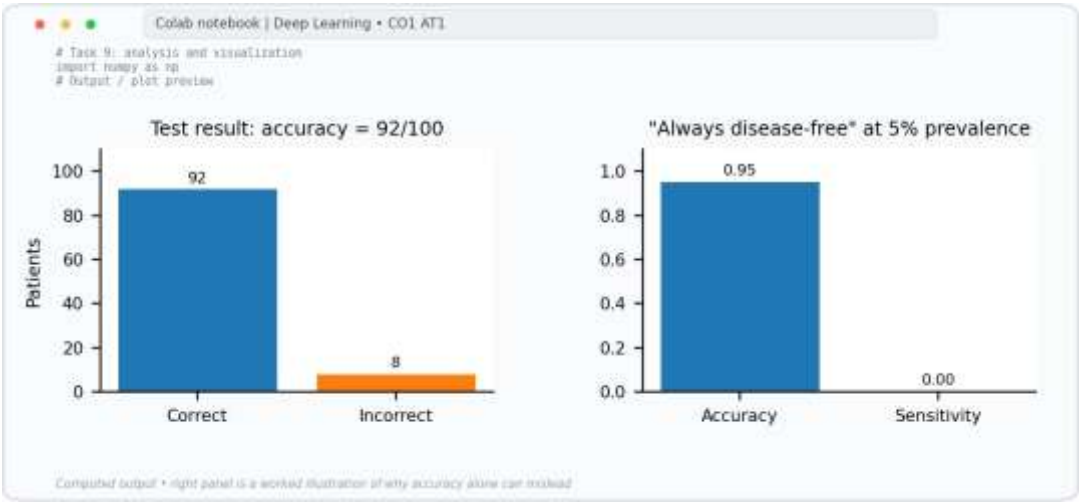
Evaluation measures

Method and analysis. Predict on the held-out test set, count agreement between predicted and true labels, and compute accuracy, then break the errors down into false positives and false negatives with a confusion matrix.

Interpretation. In healthcare a missed case (false negative) usually costs more than a false alarm, so the operating threshold should be chosen from clinical harms and capacity rather than by maximizing accuracy alone, and confidence intervals should be reported.

Quantity	Formula	Value
Accuracy	(TP + TN)/N	92/100 = 0.92
Error rate	(FP + FN)/N	8/100 = 0.08
Sensitivity (recall)	TP/(TP + FN)	needs confusion matrix
Specificity	TN/(TN + FP)	needs confusion matrix
"Always disease-free" at 5% prevalence	accuracy; sensitivity	0.95; 0.00

```
y_pred = model.predict(X_test)
accuracy = (y_pred == y_test).mean()          # 92 / 100 = 0.92
print(accuracy, confusion_matrix(y_test, y_pred))
print(recall_score(y_test, y_pred))           # sensitivity
```



TASK 10 — Unsupervised learning and patient grouping

(a) The records contain age, blood pressure, glucose and cholesterol but no risk labels, so there is no target to learn; unsupervised learning can still find structure in the features, for example groups of patients with similar profiles. **(b)** K-means picks K initial centroids (k-means++ is common), assigns every patient to the nearest centroid, recomputes each centroid as the mean of its members, and repeats until assignments stop changing, minimizing $J = \Sigma_i \|x_i - \mu_{c(i)}\|Z$. Several random starts are compared because the result can be a local minimum, and K is chosen with an elbow curve or silhouette score. **(c)** Similarity is measured with Euclidean distance on normalized features: a patient joins the cluster whose centroid is closest. For example, x = (0.70, 0.80, 0.60) with centroids c_1 = (0.30, 0.40, 0.35) and c_2 = (0.65, 0.75, 0.55) gives $d_1 = \sqrt{0.3825} = 0.6185$ and $d_2 = \sqrt{0.0075} = 0.0866$, so the patient is assigned to cluster 2. **(d)** Cluster centroids and distributions can be summarized for clinicians, who may then investigate why a group has high glucose and cholesterol and whether it deserves outreach or extra screening. Clusters are hypotheses, not diagnoses, and must be checked for stability and bias. In the simulated cohort of 600 patients, K = 3 on standardized features gives clusters of sizes 191, 215 and 194 and WCSS 556.2.

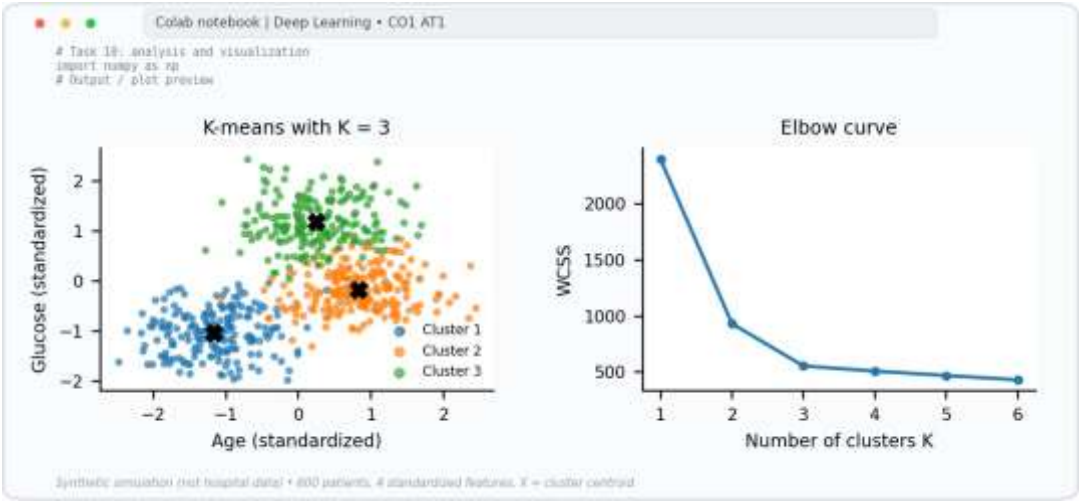
One K-means assignment step

Method and analysis. Standardize the features, run K-means with several initializations, inspect the WCSS curve, then profile each cluster by its centroid, spread and size before showing it to clinicians.

Interpretation. K-means needs K in advance, is sensitive to scale and outliers, and assumes compact, roughly spherical clusters. Cluster names such as “high-glucose group” are descriptive only and must not replace clinical assessment.

Centroid	Coordinates	Squared distance	Distance to x
c1	(0.30, 0.40, 0.35)	0.3825 (0.1600 + 0.1600 + 0.0625)	0.6185
c2	(0.65, 0.75, 0.55)	0.0075 (0.0025 × 3)	0.0866 (assign)

```
Z = StandardScaler().fit_transform(X_patients)
km = KMeans(n_clusters=3, n_init=10, random_state=42).fit(Z)
print(np.bincount(km.labels_), km.inertia_)
print(km.cluster_centers_)                    # profile each cluster for clinicians
```



TASK 11 — Comparison of supervised and unsupervised learning

(a) Dataset A contains features with disease labels, so **supervised learning** (classification) is suitable: the labels define the target and allow training and testing of predictions. Dataset B has features only, so **unsupervised learning** (clustering or dimensionality reduction) is suitable: it can reveal structure but cannot know which pattern means disease. **(b)** Supervised learning minimizes prediction error against known labels and outputs a class, probability or risk score evaluated with accuracy, sensitivity, specificity and calibration. Unsupervised learning seeks groups or low-dimensional structure and outputs cluster assignments, centroids or embeddings evaluated with WCSS, silhouette, stability and clinical usefulness. **(c)** To predict disease status for a new patient, recommend a supervised classifier (for example calibrated logistic regression) trained on Dataset A, because it directly optimizes the required target and can be validated. Clustering alone does not say which cluster is diseased; it is a useful complement for exploring Dataset B. In the simulation (2,000 synthetic patients) the logistic regression reaches 0.723 test accuracy, whereas K-means with K = 2, even with the best possible cluster-to-label mapping, reaches 0.548, barely above the majority-class baseline of 0.527.

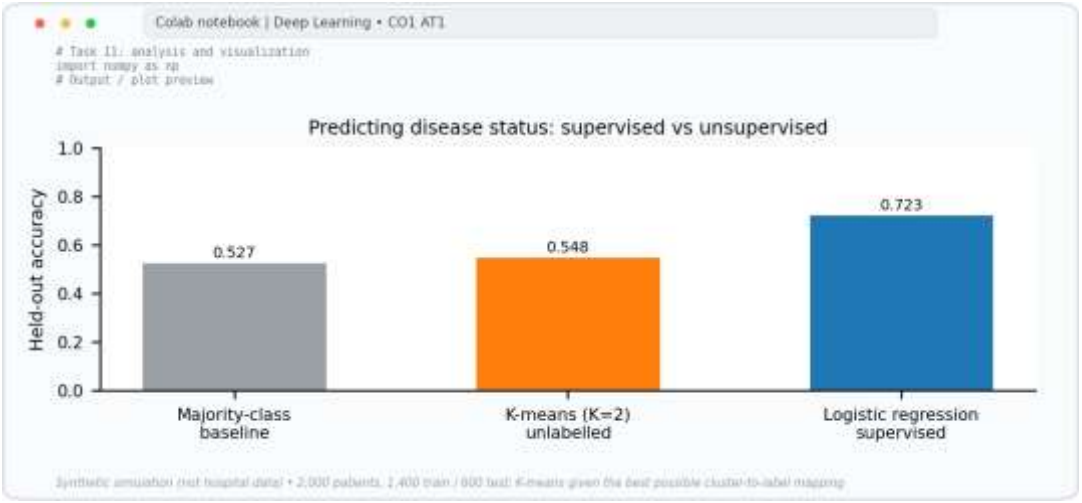
Decision evidence

Method and analysis. Both methods are run on the same synthetic features: the supervised model sees the labels during training, while K-means ignores them and its clusters are compared with the labels only afterwards for evaluation.

Interpretation. Labels are what make disease prediction possible. A hybrid workflow is sensible: train and calibrate the classifier, cluster all records separately, and compare cluster profiles with outcomes to generate hypotheses.

Aspect	Supervised (Dataset A)	Unsupervised (Dataset B)
Input	Features X and labels y	Features X only
Objective	Predict a known outcome	Discover groups or structure
Output	Class, probability or risk score	Clusters, centroids, embeddings
Evaluation	Held-out metrics, calibration	Stability, silhouette, expert review

```
clf = LogisticRegression().fit(X_train, y_train)          # supervised: uses labels
km = KMeans(n_clusters=2, n_init=10).fit(X_train)         # unsupervised: ignores labels
print(clf.score(X_test, y_test))
print((km.predict(X_test) == y_test).mean())              # compare, up to label swap
```



TASK 12 — Design challenge: selecting and adapting a learning strategy

Decision: retain the supervised baseline, modify the preprocessing and computation, and extend the design with an unsupervised branch. **A. Impact.** (a) With n = 1,000,000 and d = 4, a float64 matrix needs $10^6 \times 4 \times 8$ bytes = 32 MB, 100 times the 0.32 MB of 10,000 records, and real pipelines with hundreds of features and intermediate copies need far more. One pass of a linear model costs O(nd) and K-means costs O(nKdI), so work grows roughly 100-fold and requires batched or mini-batch processing; the sampling error of estimates shrinks only as $1/\sqrt{n}$ (about 10 times smaller). (b) Normalization keeps distances and gradients balanced; missing values must be quantified and imputed (for example median plus a missingness indicator, fitted on training data only) because dropping rows can bias results; and larger data also brings more outliers, unit errors and site differences. **B. Evaluation.** (a) One supervised model predicts only its labelled target and cannot discover patient groups that do not follow the label. (b) Labelled records may be a small, selected, noisy or delayed subset, so useful structure in the unlabelled records is ignored.

Original and redesigned strategy

Method and analysis. C. Redesign. (a) Clean and validate the data, split by patient, fit the imputer and scaler on the training split, then run two branches: a supervised classifier estimating P(D | x) and a mini-batch K-means clustering of all scaled records. (b) Probability gives calibrated risk, Euclidean distance on normalized features gives cluster similarity, normalization makes both fair across units, and capacity is raised from logistic regression to tree ensembles only if validation error improves.

Interpretation. D. Defence. (a) Compare original and redesigned pipelines on the same held-out patients using ROC-AUC, PR-AUC, sensitivity, specificity, Brier score, calibration, WCSS, silhouette and stability, plus runtime and memory at 10,000, 100,000 and 1,000,000 records (measured here: logistic regression 0.007, 0.040, 0.467 s; mini-batch K-means 0.027, 0.034, 0.161 s). (b) The design matches each objective to a suitable paradigm, controls generalization through patient-level splits, validation-based capacity and training-only preprocessing, and its cost grows about linearly with n.

Component	Original (10,000 records)	Redesigned (1,000,000 records)
Preprocessing	Complete cases, raw features	Training-fitted imputation and scaling
Prediction	Single supervised model	Calibrated classifier, capacity tuned on validation data
Patient groups	Not addressed	Mini-batch K-means on all scaled records
Computation	In-memory, full batch	Mini-batch and sampled benchmarks
Evaluation	Accuracy	AUC, sensitivity, calibration, stability, runtime

```
imp = SimpleImputer(strategy="median", add_indicator=True); sc = StandardScaler()
Xp = sc.fit_transform(imp.fit_transform(X_train))          # fit on training split only
clf = LogisticRegression(max_iter=200).fit(Xp, y_train)
km = MiniBatchKMeans(n_clusters=3, batch_size=4096, n_init=3).fit(sc.transform(imp.transform(X_all)))
```

